

Universidad Técnica Estatal de Quevedo (UTEQ)

Facultad de Ciencias de la Computación

Carrera de Ingeniería de Software (Rediseño)

Asignatura: Aplicaciones Web

Período académico: 2026–2027

Sistema de Gestión Bibliotecaria Web (SGB – SaaS)

TA – Proyecto Fin de Curso: Entrega 1A – Planificación y Diseño

Integrantes:

Cajas Ibarra Irvin Marcelo C.I.: 1251039606

Loor Medranda Marlon Taylor C.I.: 0928087469

Panama Murillo Moises Antonio C.I.: 1251334544

Docente: Ing. Gleiston Cicerón Guerrero Ulloa

Fecha de Entrega: Junio de 2026

URL del repositorio: <https://github.com/mloorm14/sgb-saas.git>

1. Descripción del sistema web propuesto

¿Qué problema resuelve? En el contexto ecuatoriano, las bibliotecas institucionales, escolares y municipales enfrentan un desafío estructural de modernización tecnológica. El análisis de requisitos del semestre anterior (2025 SPA) identificó problemáticas críticas: registros manuales de préstamos que generan aglomeración en ventanilla, un inventario opaco donde el lector no puede verificar disponibilidad sin desplazarse físicamente, y cálculos inexactos de sanciones moratorias. Investigaciones recientes confirman que estos retos son comunes en el sector bibliotecario latinoamericano [1, 2].

¿Quiénes son los usuarios? El sistema define tres roles principales:

- **Lector (Estudiante / Público general):** Accede al catálogo web, consulta disponibilidad, gestiona reservas, favoritos y préstamos activos.
- **Bibliotecario:** Registra transacciones de préstamo y devolución desde la ventanilla web con buscador predictivo.
- **Gerente / Administrador:** Gestiona usuarios, parámetros del sistema y analiza reportes estadísticos de uso de la colección.

¿Por qué una aplicación web? El SGB (Sistema de Gestión Bibliotecaria) es una plataforma SaaS (*Software as a Service*) 100 % web que elimina la necesidad de instalar software local o adquirir hardware especializado: basta un navegador moderno y conexión a internet para acceder desde cualquier dispositivo [3]. La arquitectura web permite la autogestión del estudiante desde casa, centraliza los datos con respaldo continuo y facilita la escalabilidad a múltiples instituciones sobre la misma plataforma. Además, la integración de la Google Books API automatiza el registro del inventario mediante ISBN, eliminando el ingreso manual de metadata bibliográfica [1]. Un asistente virtual basado en **Gemini 2.0 Flash API** (IA generativa) amplía la atención al usuario a las 24 horas, respondiendo consultas en lenguaje natural sobre disponibilidad, políticas y recomendaciones de lectura [4, 5].

¿Cuál es el alcance de esta entrega? Se implementará el núcleo funcional transaccional: (1) módulo de autenticación seguro con JWT; (2) CRUD del inventario bibliográfico con autocompletado ISBN; (3) módulo web de préstamos, devoluciones, reservas y multas con cálculo dinámico; y (4) chatbot con IA generativa integrado al panel del lector. Quedan fuera del alcance: pasarelas de pago electrónico, control de accesos físicos a instalaciones, escaneo de códigos de barras o QR físicos, y aplicaciones móviles nativas. El diseño de la base de datos contempla estas extensiones para desarrollo futuro.

2. Revisión y actualización de requisitos del semestre anterior

2.1. Resumen del trabajo previo

Durante el semestre 2025 SPA, el equipo ejecutó un proceso de elicitación de requisitos apoyado en entrevistas estructuradas y observación directa del flujo operativo de la biblioteca [6, 7]. Se documentaron **13 requisitos funcionales** (RF-01 a RF-13) y **6 requisitos no funcionales** (RNF-01 a RNF-06) en un artefacto SRS (*Software Requirements Specification*) siguiendo prácticas ágiles orientadas a Scrum [8]. El análisis modeló el ciclo completo de préstamos, devoluciones, reservas, multas y auditoría. Las herramientas utilizadas fueron: Google Docs (documentación), draw.io (diagramas de flujo) y GitHub (control de versiones).

2.2. Tabla de adaptación de requisitos

Para cada requisito funcional del semestre anterior se indica su estado de transición al contexto web: **Mantiene** (válido tal cual), **Adapta** (reformulado para web), **Divide** (descompuesto en sub-requisitos) o **Fuera de alcance**.

Cód.	Requisito original (2025 SPA)	Estado	Adaptación para la aplicación web
RF-01	Gestión de Identidad: validar credenciales contra BD para autorizar préstamo y confirmar pertenencia institucional.	Adapta	El mecanismo de QR institucional se sustituye por un formulario web con email y contraseña. El backend emite un JWT firmado (HS256) como token de sesión. Genera RF-01 web .
RF-02	Gestión de Préstamos: registrar inicio del préstamo mediante escaneo de QR; como contingencia, ingreso manual del número de identificación.	Adapta	Se elimina el escaneo físico de QR. El proceso se realiza mediante formulario web con buscador predictivo de usuarios y libros. Genera RF-04 web .
RF-03	Registro de Tiempos: capturar automáticamente fecha y hora en entrega y devolución.	Mantiene	Los <i>timestamps</i> se capturan automáticamente en el backend mediante <code>TIMESTAMP DEFAULT NOW()</code> en PostgreSQL. Sin cambios conceptuales; incluido en RF-04 web.
RF-04	Liberación de Material: calcular tiempo transcurrido hasta la devolución y cambiar estado a Disponible.	Mantiene	La lógica de cambio de estado y liberación del stock se implementa en el backend Spring Boot como parte de RF-05 web (devoluciones).
RF-05	Consultar Disponibilidad: permitir al estudiante verificar ejemplares en estantería antes de acudir al mostrador.	Mantiene	Se implementa como catálogo público web con indicador de stock en tiempo real. Genera RF-03 web .

Continúa en la siguiente página

Cód.	Requisito original (2025 SPA)	Estado	Adaptación para la aplicación web
RF-06	Asistente Virtual (NLP): responder consultas 24 h con precisión $\geq 90\%$.	Adapta	Se sustituye la solución NLP basada en <i>intents</i> por la API Gemini 2.0 Flash (IA generativa real), con comprensión de lenguaje natural sin programación de respuestas predefinidas [4, 5]. Genera RF-06 web .
RF-07	Gestión de Reservas y Caducidad: reservar material vía asistente virtual; cancelar reservas no retiradas en ≤ 2 horas.	Adapta	La reserva se realiza mediante formulario web dedicado. La caducidad automática se implementa como servicio programado (Spring @Scheduled). Genera RF-07 web .
RF-08	Renovación de Préstamo: renovar vía asistente virtual sin sanciones ni reservas activas de otros usuarios.	Adapta	Se implementa como función accesible desde el panel web del lector, con las mismas restricciones de negocio. Incluido en RF-04 web .
RF-09	Búsqueda Predictiva: sugerir títulos y autores en tiempo real con respuesta < 2 s.	Mantiene	Se implementa en Angular con <i>debounce</i> y llamadas al endpoint REST del backend. Tiempo de respuesta < 2 s. Incluido en RF-03 web .
RF-10	Alertas de Vencimiento: notificaciones por App / correo institucional con ≥ 15 min de anticipación.	Adapta	Las alertas se presentan como notificaciones visuales en el dashboard web del lector. El canal App / correo queda fuera del alcance de este semestre. Genera RF-08 web .
RF-11	Cálculo de Multas: calcular sanción automáticamente y bloquear al usuario hasta regularización.	Mantiene	El algoritmo ($\text{días_retraso} \times \text{tarifa_diaria}$) se implementa en el backend con tarifa configurable en <code>configuracion_sistema</code> . Genera RF-05 web .
RF-12	Reportes Estadísticos: el Gerente genera reportes de materiales solicitados, morosidad y patrones de uso.	Adapta	Los reportes se presentan como gráficas interactivas renderizadas en Angular (Chart.js / ng2-charts), accesibles desde el panel del Gerente. Genera RF-09 web .
RF-13	Registro de Auditoría (Bitácora): registrar todas las transacciones con usuario, fecha, hora y acción.	Mantiene	Se implementa como tabla <code>bitacora_auditoria</code> <i>append-only</i> en PostgreSQL, enriquecida con campos <code>ip_origen</code> y tipos de operación expandidos. Genera RF-10 web .

Cuadro 1: Tabla de adaptación de los 13 requisitos funcionales del semestre anterior.

2.3. Nuevos requisitos específicos del contexto web

Los siguientes requisitos no existían en el semestre anterior y son necesarios por la naturaleza web de la solución:

- **RNF-WEB-01:** La aplicación debe ser accesible desde navegadores modernos (Chrome 120+, Firefox 120+, Edge 120+) sin plugins adicionales.
- **RNF-WEB-02:** El intercambio de datos entre Angular y el backend debe realizarse en formato JSON con la estructura estándar `{success, data, message, errors, meta}`.

- **RNF-WEB-03:** Todas las comunicaciones entre cliente y servidor deben realizarse sobre **HTTPS** (TLS 1.2 o superior) para garantizar confidencialidad e integridad [9].
- **RF-WEB-01:** Al registrar un nuevo ejemplar, el sistema debe consumir la Google Books API mediante el ISBN para autocompletar metadata bibliográfica sin intervención manual.
- **RF-WEB-02:** Los endpoints de la API REST deben estar documentados de forma interactiva con Swagger / OpenAPI 3.0, accesibles en `/api/docs`.

3. Lista consolidada de requisitos funcionales

ID	Descripción	Prior.	Criterio de aceptación
RF-01	Autenticación segura mediante JWT: el sistema valida email y contraseña (BCrypt), emite un token JWT firmado (HS256) con el rol del usuario y gestiona logout mediante lista negra de tokens (<code>tokens_invalidos</code>).	Alta	El acceso a rutas protegidas devuelve HTTP 401 sin un JWT válido; el token invalidado por logout es rechazado en subsecuentes peticiones.
RF-02	CRUD del inventario bibliográfico con autocompletado ISBN: al ingresar un ISBN, el backend consulta la Google Books API y devuelve título, autores, editorial, resumen y URL de portada; el bibliotecario confirma y ajusta el stock [1].	Alta	Al ingresar un ISBN válido, los campos del formulario se autocompletan en menos de 3 s; al guardar, el libro aparece en el catálogo con portada visible.
RF-03	Catálogo público con búsqueda predictiva: vista accesible sin autenticación que muestra portadas, título, autor y un indicador de disponibilidad (verde: <code>stock_disponible</code> >0 / rojo: agotado); el buscador sugiere títulos y autores en tiempo real con respuesta <2 s.	Alta	Los resultados de una búsqueda por título, autor o categoría se muestran en pantalla en menos de 2 s bajo carga normal; el indicador de disponibilidad refleja el valor real de <code>stock_disponible</code> .
RF-04	Registro y renovación de préstamos: el bibliotecario registra préstamos mediante formulario web con buscador predictivo de usuarios y libros; el lector logueado puede renovar un préstamo activo si no tiene sanciones vigentes ni reservas de terceros sobre el mismo título.	Alta	Al confirmar el préstamo, <code>stock_disponible</code> decrementa en 1 y el estado del préstamo es ACTIVO ; la renovación se rechaza con mensaje explícito si existen sanciones o reservas activas de terceros.

Continúa en la siguiente página

ID	Descripción	Prior.	Criterio de aceptación
RF-05	Registro de devoluciones y cálculo dinámico de multas: al registrar una devolución tardía, el backend calcula automáticamente la sanción ($\text{días_retraso} \times \text{monto_multa_diaria de configuracion_sistema}$) y bloquea al usuario para nuevos préstamos hasta su regularización.	Alta	La devolución tardía muestra el monto exacto en pantalla; el usuario queda en estado BLOQUEADO_POR_MULTA y no puede registrar nuevos préstamos hasta que la multa esté en estado PAGADA .
RF-06	Asistente virtual con IA generativa: chatbot <i>widget</i> flotante disponible 24 h para usuarios logueados, <i>powered by Gemini 2.0 Flash API</i> ; responde consultas en lenguaje natural sobre disponibilidad, políticas, horarios y recomendaciones de lectura [4, 10].	Media	El chatbot responde consultas libres sobre la biblioteca en menos de 5 s; la clave de API de Gemini nunca es expuesta en el código del frontend.
RF-07	Sistema de reservaciones web con caducidad automática: el lector puede reservar un título actualmente prestado; el sistema cancela automáticamente la reserva si no es recogida en ventanilla en el plazo configurado.	Media	Una reserva en estado PENDIENTE cambia a EXPIRADA al vencer el plazo; el lector recibe una alerta visual en su panel indicando la caducidad.
RF-08	Alertas de vencimiento en dashboard: el sistema muestra notificaciones visuales en el panel del lector cuando un préstamo está próximo a vencer (según $\text{dias_aviso_vencimiento de configuracion_sistema}$) o ha expirado.	Media	Los préstamos con fecha de devolución dentro del umbral configurado aparecen destacados en el panel del lector con indicador de color (amarillo / rojo).
RF-09	Reportes estadísticos interactivos para el Gerente: panel con gráficas sobre materiales más solicitados, índice de morosidad y patrones de uso de la colección, para apoyar decisiones de adquisición [11].	Baja	El Gerente visualiza al menos 3 reportes gráficos (top libros, morosidad, actividad mensual) con datos del período seleccionado; el acceso es exclusivo del rol ROLE_GERENTE .
RF-10	Bitácora de auditoría inmutable: toda operación crítica (préstamos, devoluciones, multas, cambios de inventario, intentos de login) se registra automáticamente en bitacora_auditoria con usuario, tipo de operación, tabla afectada, registro_id (NULL para eventos de sesión), detalles e IP de origen.	Alta	Cada creación de préstamo o devolución inserta un registro en bitacora_auditoria ; los intentos de login fallidos generan un registro con usuario_id = NULL y tipo_operacion = LOGIN_FAIL .

Cuadro 2: Lista consolidada de 10 requisitos funcionales web.

4. Lista consolidada de requisitos no funcionales

ID	Categoría	Descripción	Métrica de verificación
RNF-01	Usabilidad	La interfaz Angular es completamente responsiva (diseño <i>mobile-first</i>) y opera correctamente en pantallas de 320 px a 1440 px de ancho [12, 3].	Verificado con Chrome DevTools en resoluciones: 375 px (móvil), 768 px (tablet) y 1280 px (escritorio). Todos los formularios y gráficos son utilizables sin <i>scroll</i> horizontal.
RNF-02	Rendimiento	El tiempo de respuesta de los endpoints de búsqueda y consulta de disponibilidad no supera 2 s bajo una carga de 100 usuarios concurrentes en red universitaria.	Medido con Postman Runner o JMeter con 100 hilos simultáneos; el percentil P95 de latencia debe ser ≤ 2 s.
RNF-03	Fiabilidad	El sistema mantiene una disponibilidad operativa del 99,5 % durante el horario de atención de la biblioteca (07:30–16:00), asegurando la continuidad del servicio de préstamos.	Verificado durante la demostración final (semana 17); el sistema debe responder sin interrupción durante toda la defensa oral.
RNF-04	Seguridad (TLS)	Todas las comunicaciones entre el cliente Angular y el backend Spring Boot se realizan sobre HTTPS con protocolo TLS 1.2 o superior para garantizar confidencialidad e integridad de los datos [9].	Verificado con la herramienta SSL Labs o inspección del certificado en el navegador; el sitio no debe ser accesible por HTTP plano en producción.
RNF-05	Seguridad (API)	Las contraseñas se almacenan con hash BCrypt (factor 12); los endpoints están protegidos con JWT (Spring Security); los tokens invalidados se registran en <code>tokens_invalidos</code> .	Inspección directa de la columna <code>password_hash</code> en la BD (debe comenzar con <code>\$2a\$12\$...</code>); prueba de acceso con token expirado debe retornar HTTP 401.
RNF-06	Mantenibilidad	La arquitectura modular del backend (Spring Boot) permite actualizar o sustituir el modelo de IA del chatbot sin detener los servicios principales de préstamo [13].	El endpoint <code>/api/chatbot/message</code> es independiente del módulo de préstamos; el cambio de proveedor de IA requiere modificar solo la clase <code>GeminiService</code> sin afectar otros módulos.
RNF-07	Normativo (LOPDP)	El sistema cumple con la Ley Orgánica de Protección de Datos Personales (LOPDP Ecuador), solicitando consentimiento explícito al estudiante para el procesamiento de su historial de lectura [14].	El formulario de registro incluye casilla de consentimiento obligatoria; el historial de lectura no es visible para otros usuarios ni se expone en endpoints públicos.
RNF-08	Compatibilidad	La aplicación debe funcionar sin degradación en Chrome 120+, Firefox 120+ y Edge 120+, sin necesidad de plugins adicionales.	Pruebas manuales de los flujos principales (login, catálogo, préstamo) en los tres navegadores; sin errores en la consola de desarrollo.

Continúa en la siguiente página

ID	Categoría	Descripción	Métrica de verificación
RNF-09	Documentación API	La API REST expondrá documentación interactiva generada automáticamente por springdoc-openapi (Swagger / OpenAPI 3.0) en la ruta /api/docs , con mínimo 5 endpoints documentados con modelos de <i>request</i> / <i>response</i> y códigos HTTP.	El docente accede a /api/docs en el navegador durante la demostración y puede ejecutar llamadas de prueba directamente desde la interfaz Swagger UI.
RNF-10	Integridad de datos	Ninguna consulta SQL se construye por concatenación directa de cadenas; se utiliza exclusivamente Spring Data JPA como ORM. La BD impone restricciones CHECK para prevenir stock negativo y ON DELETE RESTRICT en claves foráneas críticas.	Revisión de código: ausencia de llamadas a EntityManager.createNativeQuery() con cadenas interpoladas. Prueba directa: intentar insertar un préstamo con libro de stock_disponible = 0 debe retornar error HTTP 409.

Cuadro 3: Lista consolidada de 10 requisitos no funcionales.

5. Arquitectura de la aplicación web

5.1. Diagramas de arquitectura (Modelo C4)

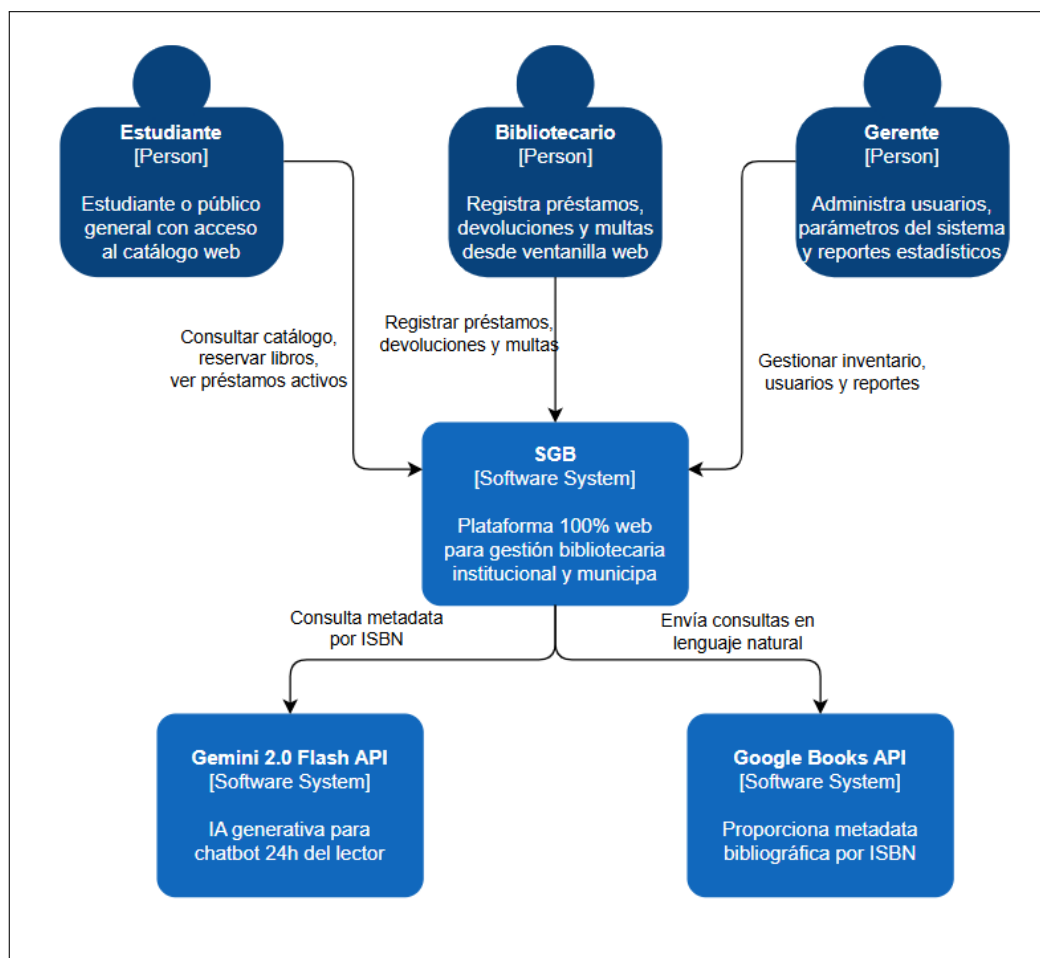


Figura 1: Diagrama C4 Nivel 1 (Contexto). Muestra el sistema SGB como caja central con sus tres tipos de usuario (Lector, Bibliotecario, Gerente) y los dos sistemas externos con los que interactúa: Google Books API (autocomplete de metadata ISBN) y Gemini 2.0 Flash API (chatbot con IA generativa). Herramienta recomendada: Structurizr DSL o draw.io.

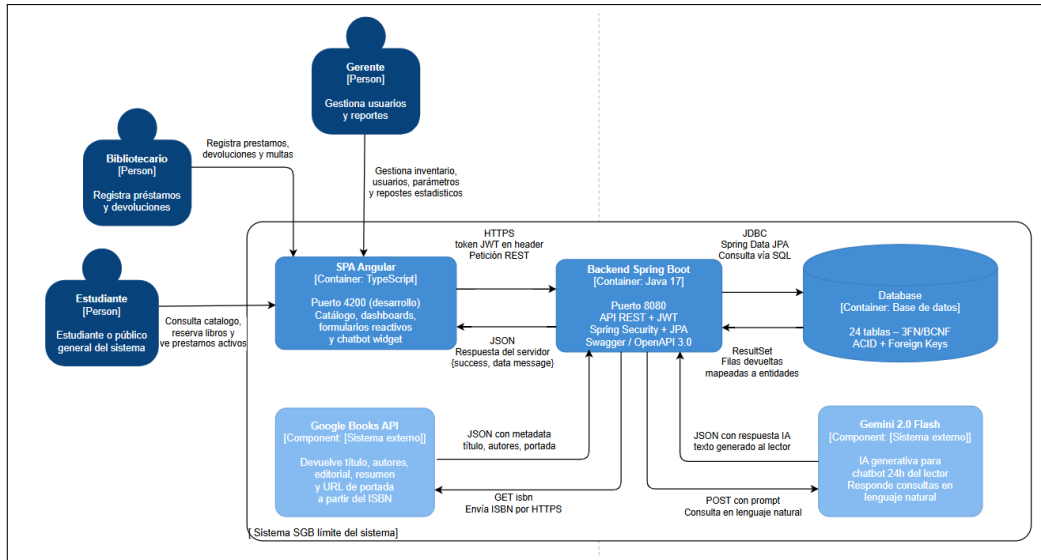


Figura 2: Diagrama C4 Nivel 2 (Contenedores). Detalla la separación en tres contenedores: (1) **SPA Angular** (TypeScript, puerto 4200 en desarrollo) – renderizado en el navegador del usuario; (2) **Backend Spring Boot** (Java 21, puerto 8080) – API REST con Spring Security, Spring Data JPA y springdoc-openapi; (3) **Base de Datos PostgreSQL 15+** – 24 tablas en 3FN/BCNF, almacenamiento persistente. Las flechas indican el protocolo de comunicación: HTTPS / JSON entre Angular y Spring Boot; JDBC / SQL entre Spring Boot y PostgreSQL [15, 16].

5.2. Descripción de capas y tecnologías

Capa	Tecnología elegida	Justificación
Frontend (SPA)	Angular 17+ / TypeScript	Framework robusto para <i>Single Page Applications</i> reactivas y modulares. Permite <i>lazy loading</i> , Guards JWT para rutas protegidas, formularios reactivos con validación en tiempo real y consumo eficiente de la API REST [17].
Estilos UI	Tailwind CSS / Bootstrap 5	Acelera el desarrollo de componentes responsivos. Tailwind para utilidades CSS; Bootstrap para componentes de formulario y modal listos para usar. Garantiza consistencia visual <i>mobile-first</i> [12].
Backend (API REST)	Spring Boot 3.x / Java 21	Framework maduro con Spring Security para implementación directa de JWT y roles, Spring Data JPA para ORM, soporte nativo de transacciones y documentación con springdoc-openapi [18].
ORM	Spring Data JPA / Hibernate 6	Abstrae la base de datos en entidades Java con anotaciones (<code>@Entity</code> , <code>@ManyToOne</code> , <code>@ManyToMany</code>). Previene inyección SQL al eliminar la concatenación directa; usa repositorios con <code>@Query</code> parametrizadas.
Base de datos	PostgreSQL 15+	Motor relacional ACID con integridad referencial via Foreign Keys y restricciones CHECK. Soporta 24 tablas en 3FN/BCNF, índices parciales y la tabla <code>bitacora_auditoria append-only</code> sin pérdida de rendimiento.
Integraciones externas	Google Books API + Gemini 2.0 Flash API	Google Books: autocompletado de metadata bibliográfica por ISBN (gratuito). Gemini 2.0 Flash: IA generativa para el chatbot con comprensión real de lenguaje natural (capa gratuita: 15 RPM, 1 500 req/día) [4, 13, 19].
Control de versiones	Git / GitHub	Repositorio compartido del equipo con historial de <i>commits</i> de todos los integrantes; ramas <code>feature/*</code> con revisión por PR.
Documentación API	Swagger UI / OpenAPI 3.0	Documentación interactiva generada automáticamente por <code>springdoc-openapi</code> ; accesible en <code>/api/docs</code> desde la Entrega 1B.

Cuadro 5: Stack tecnológico seleccionado y justificaciones técnicas.

5.3. ADR-001: Decisión de tecnología principal

Título	ADR-001: Selección del framework backend del servidor
Estado	Aceptado
Contexto	El equipo necesita un lenguaje y framework del lado del servidor seguro, escalable, con soporte para JWT, ORM integrado y documentación automática de la API. Se consideró el conocimiento previo del equipo, la disponibilidad de documentación y la madurez del ecosistema.
Opciones consideradas	Opción A: Laravel 11 (PHP 8.2) – ecosistema maduro, curva de aprendizaje media; requiere configuración adicional de JWT. Opción B: ASP.NET Core 8 (C#) – alto rendimiento; curva de aprendizaje pronunciada para el equipo. Opción C: Spring Boot 3 (Java 21) – Spring Security nativo para JWT, Spring Data JPA integrado, springdoc-openapi para Swagger.
Decisión	Se eligió Spring Boot 3 con Java 21 . Ofrece un ecosistema maduro: Spring Security para la implementación directa de JWT y roles, Spring Data JPA que elimina el SQL concatenado (requisito explícito del PFC), y documentación automática con springdoc-openapi. El equipo cuenta con conocimiento previo de Java desde cursos anteriores.
Consecuencias positivas	■ Seguridad empresarial out-of-the-box (Spring Security). ■ ORM robusto (Hibernate 6) con validación de entidades. ■ Swagger UI generado automáticamente; cero configuración manual. ■ Transaccionalidad segura con anotación <code>@Transactional</code>.
Consecuencias negativas	■ Curva de aprendizaje en filtros de Spring Security y configuración de CORS. Se mitiga con documentación oficial y tutoriales de Baeldung. ■ Mayor tiempo de arranque (<i>cold start</i>) respecto a frameworks interpretados. Aceptable para el contexto universitario.

Cuadro 7: Architecture Decision Record (ADR-001).

6. Modelo de base de datos

6.1. Modelo Entidad-Relación (MER)

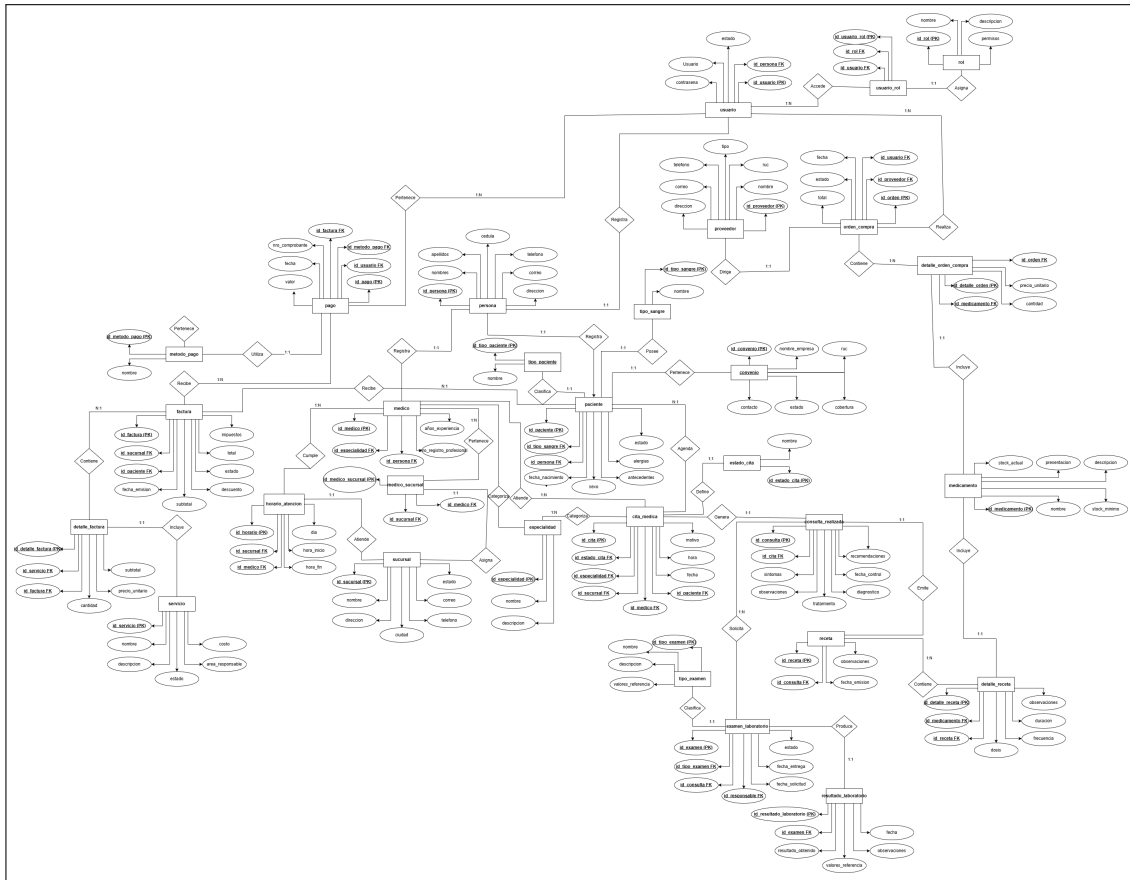


Figura 3: Diagrama MER del SGB – 24 tablas organizadas en 5 módulos: Seguridad (5), Catálogo e Inventario (8), Transacciones (7), Experiencia del Estudiante (3) y Auditoría (1). Herramienta recomendada: dbdiagram.io o MySQL Workbench. El esquema aplica 3FN/BCNF: tablas maestras de selección eliminan texto libre; tablas *junction* resuelven relaciones M:N.

6.2. Diccionario de datos

Se documenta el diccionario de datos de las 5 entidades principales. El diccionario completo de las 24 tablas con todos sus atributos se encuentra en el repositorio: docs/diccionario_datos.md.

Tabla: usuarios

Atributo	Tipo	Nulo	Restricción	Descripción
id	BIGSERIAL	No	PK, AUTO	Identificador único.
nombre	VARCHAR(100)	No	NOT NULL	Nombre de pila del usuario.
apellido	VARCHAR(100)	No	NOT NULL	Apellido(s).
correo	VARCHAR(150)	No	UNIQUE, NOT NULL	Correo de acceso.
password_hash	VARCHAR(255)	No	NOT NULL	Hash BCrypt de la contraseña.
identificacion_usuario	VARCHAR(20)	Sí	—	Identificador del usuario (Cédula/ID).
estado_id	INTEGER	No	FK → estados_usuario.id	Estado operativo.
fecha_registro	TIMESTAMP	No	DEFAULT NOW()	Alta en el sistema.

Tabla: libros

Atributo	Tipo	Nulo	Restricción	Descripción
id	BIGSERIAL	No	PK, AUTO	Identificador del título.
isbn	VARCHAR(13)	No	UNIQUE, NOT NULL	Código ISBN-13.
titulo	VARCHAR(255)	No	NOT NULL	Título completo.
resumen	TEXT	Sí	—	Sinopsis (API).
portada_url	VARCHAR(1000)	Sí	—	URL de portada (Google Books).
anio_publicacion	SMALLINT	No	CHECK (1000–2100)	Año de publicación.
editorial_id	INTEGER	No	FK → editoriales.id	Editorial del libro.
idioma_id	INTEGER	No	FK → idiomas.id	Idioma del contenido.
estado_id	INTEGER	No	FK → estados_libro.id	Estado operativo.
stock_total	SMALLINT	No	DEFAULT 1, ≥0	Ejemplares físicos totales.
stock_disponible	SMALLINT	No	DEFAULT 1, ≥0	Ejemplares en estante.
ubicacion_fisica	VARCHAR(50)	Sí	—	Referencia de estante.
fecha_registro	TIMESTAMP	No	DEFAULT NOW()	Alta en catálogo.

Tabla: prestamos

Atributo	Tipo	Nulo	Restricción	Descripción
id	BIGSERIAL	No	PK, AUTO	Identificador de la transacción.
usuario_id	BIGINT	No	FK → usuarios.id	Estudiante prestatario.
libro_id	BIGINT	No	FK → libros.id	Ejemplar prestado.
bibliotecario_id	BIGINT	No	FK → usuarios.id	Quién procesa.
reservacion_id	BIGINT	Sí	FK → reservaciones.id	Reserva origen.
fecha_prestamo	TIMESTAMP	No	DEFAULT NOW()	Inicio del préstamo.
fecha_devolucion_estimada	TIMESTAMP	No	NOT NULL	Fecha límite.
fecha_devolucion_real	TIMESTAMP	Sí	—	Devolución efectiva.
renovaciones_realizadas	SMALLINT	No	DEFAULT 0, ≥0	Conteo de renovaciones.
estado_prestamo_id	INTEGER	No	FK → estados_prestamo.id	Estado del ciclo.

Tabla: multas

Atributo	Tipo	Nulo	Restricción	Descripción
id	BIGSERIAL	No	PK, AUTO	Identificador de la multa.
prestamo_id	BIGINT	No	FK UNIQUE → prestamos.id	Relación 1:1.
monto	NUMERIC(8,2)	No	CHECK >0	Monto calculado.
estado_multa_id	INTEGER	No	FK → estados_multa.id	Estado del cobro.
fecha_generada	TIMESTAMP	No	DEFAULT NOW()	Generación automática.
fecha_pagada	TIMESTAMP	Sí	—	Regularización.
observaciones	VARCHAR(255)	Sí	—	Notas del bibliotecario.

Tabla: bitacora_auditoria

Atributo	Tipo	Nulo	Restricción	Descripción
id	BIGSERIAL	No	PK, AUTO	Identificador del evento.
usuario_id	BIGINT	Sí	FK → usuarios.id	Operador (NULL en login fallido).
tipo_operacion	VARCHAR(20)	No	CHECK (valores fijos)	Tipo de evento auditado.
tabla_afectada	VARCHAR(50)	No	NOT NULL	Entidad modificada.
registro_id	BIGINT	Sí	—	ID del registro (NULL en sesiones).
detalles	TEXT	No	NOT NULL	Descripción detallada del cambio.
ip_origen	VARCHAR(45)	Sí	—	IP del cliente (IPv4 / IPv6).
fecha_hora	TIMESTAMP	No	DEFAULT NOW()	Marca temporal del evento.

6.3. Script DDL de creación (PostgreSQL 15+)

El script siguiente es ejecutable sin modificaciones en PostgreSQL 15+. El DDL completo de las 24 tablas con índices y datos de semilla se encuentra en `database/schema.sql` del repositorio.

```
--          TABLAS MAESTRAS (prerequisito para las entidades core)

CREATE TABLE estados_usuario (
    id          SERIAL          PRIMARY KEY,
    nombre      VARCHAR(30) NOT NULL UNIQUE
    -- Valores: 'ACTIVO', 'BLOQUEADO_POR_MULTA', 'INACTIVO'
);
CREATE TABLE estados_libro (
    id          SERIAL          PRIMARY KEY,
    nombre      VARCHAR(30) NOT NULL UNIQUE
    -- Valores: 'ACTIVO', 'DADO_DE_BAJA', 'EN_REPARACION', '
    PERDIDO '
);
CREATE TABLE editoriales (
    id          SERIAL          PRIMARY KEY,
    nombre      VARCHAR(150) NOT NULL UNIQUE,
    pais_origen VARCHAR(80) NULL
);
CREATE TABLE idiomas (
    id          SERIAL          PRIMARY KEY,
    nombre      VARCHAR(50) NOT NULL UNIQUE,
    codigo_iso  VARCHAR(5) NOT NULL UNIQUE -- ISO 639-1
);
CREATE TABLE estados_prestamo (
    id          SERIAL          PRIMARY KEY,
    nombre      VARCHAR(30) NOT NULL UNIQUE
    -- Valores: 'ACTIVO', 'RENOVADO', 'DEVUELTO', 'VENCIDO'
```



```

);
CREATE TABLE estados_multa (
    id          SERIAL          PRIMARY KEY,
    nombre      VARCHAR(30) NOT NULL UNIQUE
    -- Valores: 'PENDIENTE', 'PAGADA', 'ANULADA'
);

--          ENTIDADES CORE

CREATE TABLE usuarios (
    id          BIGSERIAL        PRIMARY KEY,
    nombre      VARCHAR(100) NOT NULL,
    apellido    VARCHAR(100) NOT NULL,
    correo      VARCHAR(150) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    identificacion_usuario VARCHAR(20) NULL,
    estado_id   INTEGER          NOT NULL
    REFERENCES estados_usuario(id) ON DELETE RESTRICT,
    fecha_registro    TIMESTAMP    NOT NULL DEFAULT NOW()
);

CREATE TABLE libros (
    id          BIGSERIAL        PRIMARY KEY,
    isbn        VARCHAR(13)      NOT NULL UNIQUE,
    titulo      VARCHAR(255)     NOT NULL,
    resumen     TEXT             NULL,
    portada_url VARCHAR(1000)    NULL,
    anio_publicacion SMALLINT    NOT NULL
    CHECK (anio_publicacion BETWEEN 1000 AND 2100),
    editorial_id INTEGER          NOT NULL
    REFERENCES editoriales(id) ON DELETE RESTRICT,
    idioma_id   INTEGER          NOT NULL
    REFERENCES idiomas(id) ON DELETE RESTRICT,
    estado_id   INTEGER          NOT NULL
    REFERENCES estados_libro(id) ON DELETE RESTRICT,
    stock_total SMALLINT          NOT NULL DEFAULT 1
    CHECK (stock_total >= 0),
    stock_disponible SMALLINT    NOT NULL DEFAULT 1
    CHECK (stock_disponible >= 0),
    ubicacion_fisica VARCHAR(50) NULL,
    fecha_registro    TIMESTAMP    NOT NULL DEFAULT NOW(),
    CONSTRAINT chk_stock CHECK (stock_disponible <= stock_total)
);

CREATE TABLE prestamos (
    id          BIGSERIAL        PRIMARY KEY,
    usuario_id  BIGINT           NOT NULL
    REFERENCES usuarios(id) ON DELETE RESTRICT,
    libro_id    BIGINT           NOT NULL
    REFERENCES libros(id) ON DELETE RESTRICT,

```

```

    bibliotecario_id          BIGINT      NOT NULL
        REFERENCES usuarios(id)          ON DELETE RESTRICT,
    fecha_prestamo             TIMESTAMP NOT NULL DEFAULT NOW(),
    fecha_devolucion_estimada  TIMESTAMP NOT NULL,
    fecha_devolucion_real      TIMESTAMP NULL,
    renovaciones_realizadas    SMALLINT   NOT NULL DEFAULT 0
        CHECK (renovaciones_realizadas >= 0),
    estado_prestamo_id         INTEGER     NOT NULL
        REFERENCES estados_prestamo(id) ON DELETE RESTRICT
);

CREATE TABLE multas (
    id                          BIGSERIAL    PRIMARY KEY,
    prestamo_id                BIGINT        NOT NULL UNIQUE
        REFERENCES prestamos(id) ON DELETE RESTRICT,
    monto                      NUMERIC(8,2) NOT NULL CHECK (monto > 0),
    estado_multa_id            INTEGER        NOT NULL
        REFERENCES estados_multa(id),
    fecha_generada             TIMESTAMP     NOT NULL DEFAULT NOW(),
    fecha_pagada               TIMESTAMP     NULL,
    observaciones              VARCHAR(255)  NULL
);

CREATE TABLE bitacora_auditoria (
    id                          BIGSERIAL    PRIMARY KEY,
    usuario_id                 BIGINT        NULL
        REFERENCES usuarios(id) ON DELETE SET NULL,
    tipo_operacion             VARCHAR(20)   NOT NULL
        CHECK (tipo_operacion IN (
            'INSERT', 'UPDATE', 'DELETE',
            'LOGIN_OK', 'LOGIN_FAIL', 'LOGOUT'
        )),
    tabla_afectada             VARCHAR(50)   NOT NULL,
    registro_id                BIGINT        NULL,
    detalles                   TEXT          NOT NULL,
    ip_origen                  VARCHAR(45)   NULL,
    fecha_hora                 TIMESTAMP     NOT NULL DEFAULT NOW()
);

-- NOTA: bitacora_auditoria es APPEND-ONLY.
-- Prohibido ejecutar UPDATE o DELETE sobre esta tabla.

```

7. Diseño de interfaz: Wireframes

SGB
Sistema de Gestión Bibliotecaria

Bienvenido de vuelta

Ingresa tus credenciales para acceder al sistema

CORREO ELECTRÓNICO

CONTRASEÑA [¿Olvidaste tu contraseña?](#)

☐

☐ Acepto la **Política de Privacidad** y el tratamiento de mis datos personales conforme a la **LOPD** Ecuador. (Obligatorio)

Iniciar sesión →

[¿No tienes cuenta?](#)

Sesión protegida con JWT (HS256) Sin almacenamiento en localStorage

UTEQ · Facultad de Ciencias de la Computación · 2026

Figura 4: **Pantalla de Login** (visible para todos los visitantes). Formulario centrado con campos Email y Contraseña, botón *Iniciar sesión*, enlace *Crear cuenta* y aviso de privacidad. Flujo: página de inicio → login → dashboard según rol (Bibliotecario / Gerente) o panel del Lector. El JWT recibido se almacena en memoria; no se usa `localStorage` por riesgo de XSS.

SGB
Sistema de Gestión Bibliotecaria

Crear cuenta

Regístrate para acceder al catálogo y gestionar tus préstamos

NOMBRE
Ej: Marlon
Ingresa tu nombre de pila

APELLIDO
Ej: Llor
Ingresa tu apellido

CORREO ELECTRÓNICO
ejemplo@correo.com
Puede ser cualquier correo válido

CÉDULA / DOCUMENTO DE IDENTIDAD (opcional)
Ej: 1234567890
Cédula, pasaporte u otro documento

CONTRASEÑA
Minimo 8 caracteres

CONFIRMAR CONTRASEÑA
Repite tu contraseña
Debe coincidir con la contraseña

☐ Acepto la [Política de Privacidad](#) y el tratamiento de mis datos personales conforme a la **LOPD Ecuador**. Mi historial de lectura no será compartido con terceros. (Obligatorio)

Crear cuenta →

¿Ya tienes cuenta? [Iniciar sesión](#)

Figura 5: **Pantalla de Registro** (visible para visitantes anónimos). Formulario con campos: Nombre, Apellido, Correo, Cédula o Documento de identidad (opcional), Contraseña, Confirmar contraseña y casilla de consentimiento LOPDP (obligatoria). Validación en tiempo real: formato de correo, mínimo 8 caracteres para contraseña, coincidencia de contraseñas. Al registrarse exitosamente, redirige a la pantalla de Login.

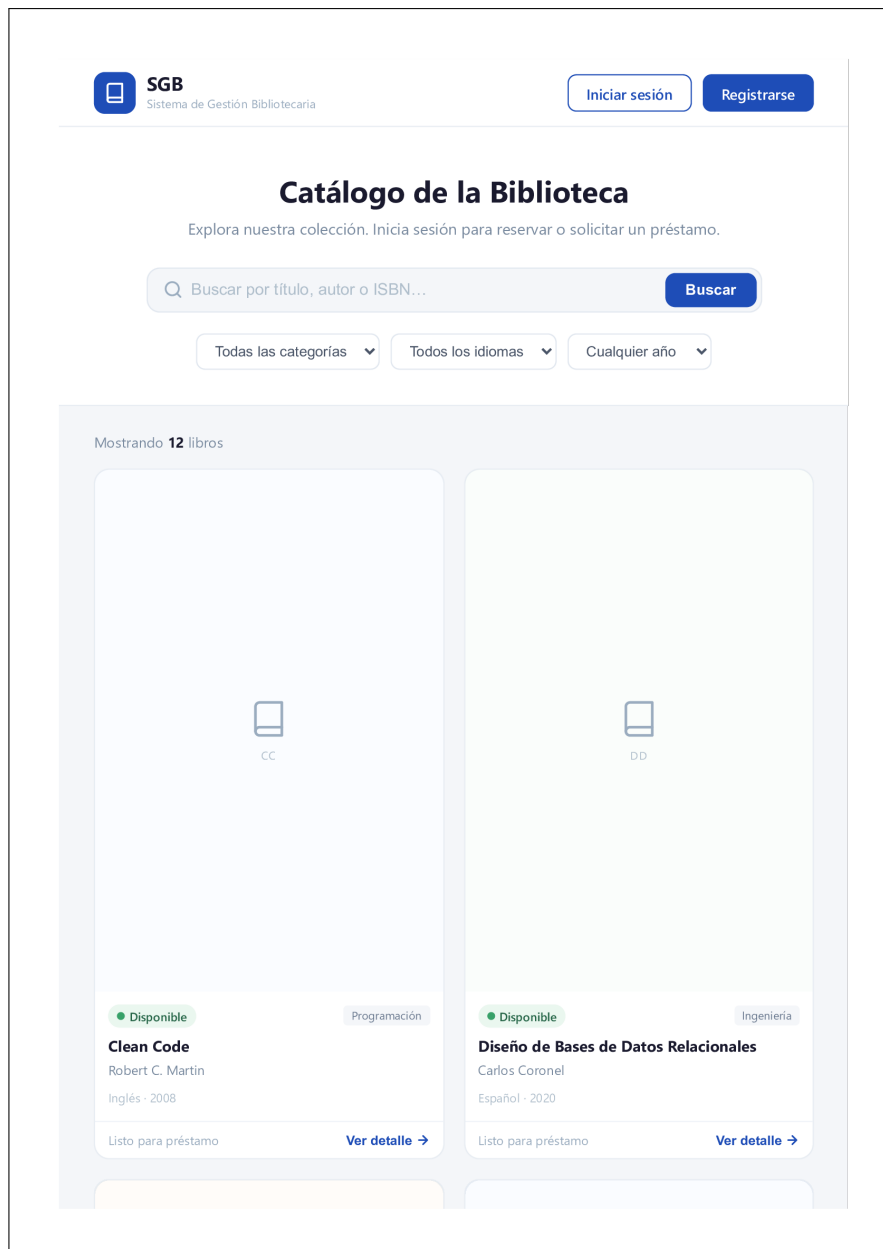


Figura 6: **Catálogo Público (Vista Home)** (accesible sin login). Barra superior con logo SGB y botones *Iniciar sesión* / *Registrarse*. Campo de búsqueda central con filtros por categoría, idioma y año. Grilla de tarjetas (4 columnas desktop / 2 móvil) mostrando portada, título, autor y *badge* de disponibilidad (● Disponible / ● Agotado). Flujo: página de inicio → detalle del libro → login para reservar.

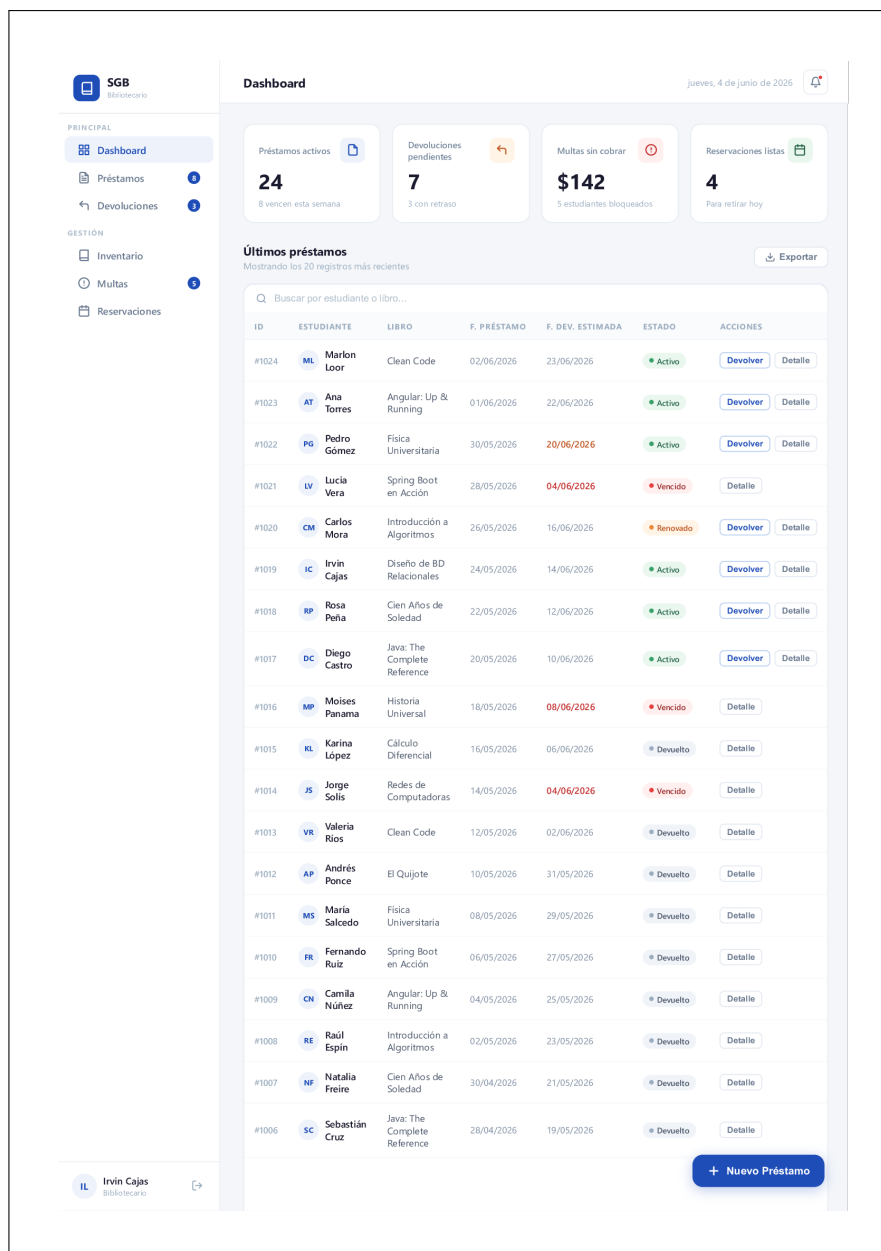


Figura 7: **Dashboard del Bibliotecario** (acceso exclusivo `ROLE_BIBLIOTECARIO`). Panel lateral izquierdo con menú: Dashboard, Préstamos, Devoluciones, Inventario, Multas, Reservaciones. Panel central superior: 4 tarjetas métricas (préstamos activos, devoluciones pendientes, multas sin cobrar, reservaciones listas). Tabla de últimos 20 préstamos con columnas: ID, Estudiante, Libro, Fecha préstamo, Fecha devolución estimada, Estado. Botón flotante *Nuevo Préstamo* abre modal con buscador predictivo de usuario y libro.

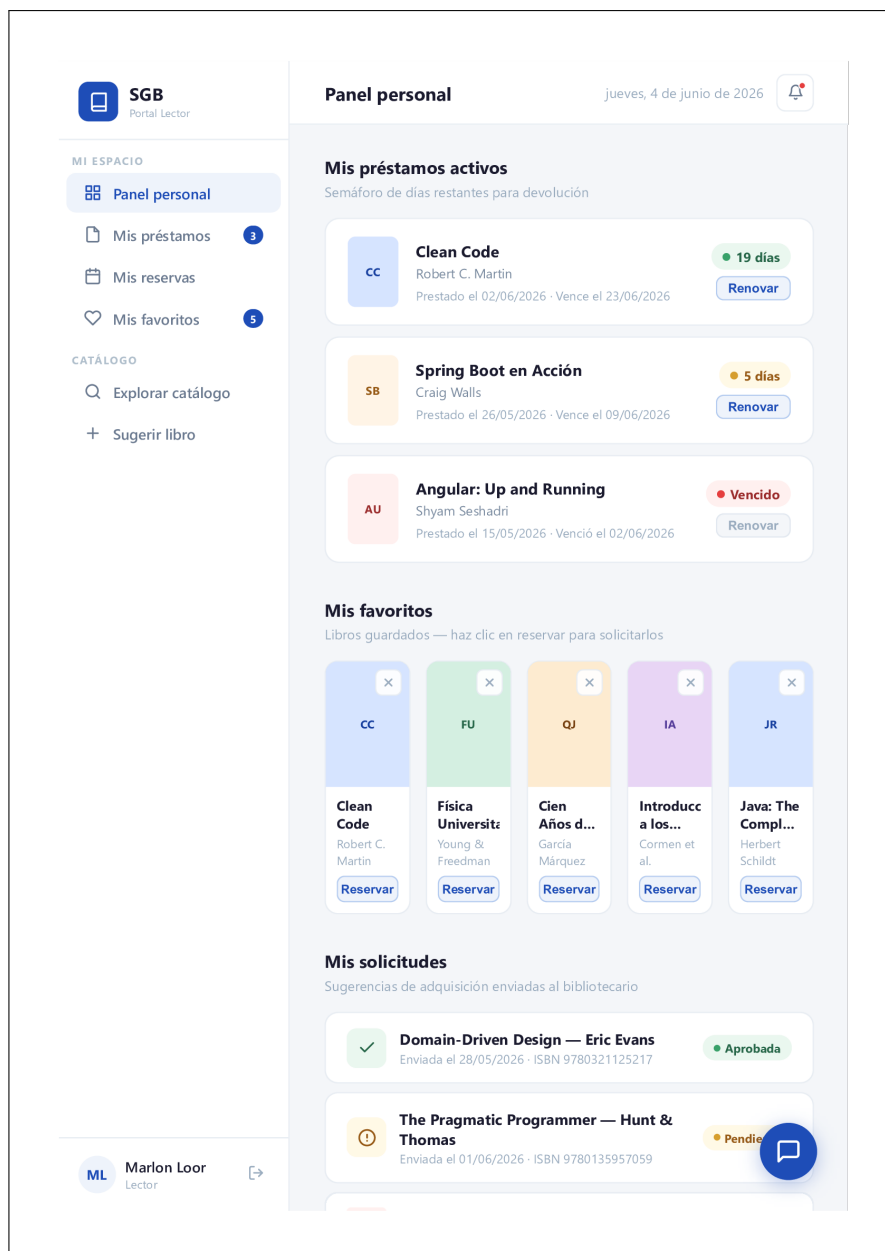


Figura 8: **Panel Personal del Lector** (acceso exclusivo `ROLE_LECTOR`). Sección *Mis Préstamos Activos* con semáforo de días restantes (verde / amarillo / rojo). Sección *Mis Favoritos* en grilla con botón de eliminar. Sección *Mis Solicitudes* con estado de cada sugerencia enviada. Widget flotante en esquina inferior derecha: Asistente Virtual Gemini, desplegable como panel de chat con historial de conversación. La clave API de Gemini se gestiona exclusivamente en el backend.

8. Plan de trabajo: cronograma e hitos

8.1. Cronograma semanal (semanas 5–17)

Semana	Tarea principal	Responsable	Entregable asociado
5	Entrega 1A: redacción del documento de planificación y diseño completo	Todo el equipo	PDF Entrega 1A en LMS (Sumativa #5)
6	Configuración de Spring Boot, PostgreSQL, Spring Data JPA y estructura del proyecto; primeras rutas REST y configuración de CORS	Marlon	Repositorio con estructura inicial en GitHub
7	PE Unidad II: desarrollo backend con Spring Boot; controladores <code>@RestController</code> , modelos JPA y endpoints básicos	Cajas	Práctica Experimental U2 (Sumativa #6)
8	Tutoría – consolidar avances, resolver <i>blockers</i> de configuración JWT y revisar integración Angular–Spring	Todo el equipo	—
9	Evaluación Parcial 1 (individual); sin desarrollo grupal esta semana	Individual	EP Primer Corte (Sumativa #7)
10	Módulo de autenticación completo con JWT (login, logout, lista negra); CRUD de entidad <code>libros</code> vía JPA con autocompletado ISBN	Marlon / Cajas	Avance PFC – Módulo datos (Sumativa #8)
11	CRUD completo de la entidad principal <code>prestamos</code> con ORM; endpoints de devolución y cálculo de multas	Cajas	PE Unidad III (Sumativa #9)
12	Integración de capas frontend–backend; pruebas del módulo transaccional; corrección de bugs de integración	Todo el equipo	—
13	Desarrollo del frontend Angular responsivo: catálogo público, formularios de login / registro y dashboard del bibliotecario	Panamá	Exposición Back-End (Sumativa #12)
14	Implementación MVC completa con API REST; integración del chatbot Gemini 2.0 Flash; configuración de Swagger en <code>/api/docs</code>	Todo el equipo	—
15	Entrega 1B: sistema funcionando con JWT + CRUD + ORM; documentación del repositorio Git y README ejecutable	Todo el equipo	Entrega 1B en LMS (Sumativa #15)
16	PE Unidad IV: pruebas de aceptación con Postman; validación de endpoints documentados en Swagger; correcciones finales de rendimiento y seguridad	Todo el equipo	PE Unidad IV (Sumativa #16)
17	Defensa oral del PFC: demostración en vivo del sistema completo; informe técnico final (norma APA, resumen bilingüe, comparativa SOAP vs. REST, reflexión sobre IA generativa)	Todo el equipo	EP Segundo Corte + Defensa (Sumativa #17)

Cuadro 13: Cronograma semanal del proyecto – semanas 5 a 17.

8.2. Asignación de roles del equipo

Integrante	Rol principal	Responsabilidades
Loor Medranda Marlon Taylor	Líder Backend / DevOps	Configuración Spring Boot, base de datos PostgreSQL, seguridad JWT, módulo ISBN / Google Books API, despliegue y pruebas Postman.
Cajas Ibarra Irvin Marcelo	Desarrollador Backend	Modelos JPA y entidades, controladores REST, lógica transaccional de préstamos / devoluciones, cálculo de multas y documentación Swagger.
Panama Murillo Moises Antonio	Desarrollador Frontend	Desarrollo SPA Angular, consumo de APIs REST, diseño responsivo, integración del chatbot Gemini y UX / UI.

Cuadro 15: Roles y responsabilidades del equipo.

9. Estructura del repositorio Git

El repositorio está disponible en GitHub en la URL indicada en la portada. Al momento de la Entrega 1A, todos los integrantes del equipo tienen al menos 1 *commit* registrado. La estructura mínima de directorios es la siguiente:

```
/
|-- README.md                <- Descripción, instalación, integrantes, URL deploy
|-- .gitignore               <- Excluye: .env, node_modules/, target/, *.log
|-- .env.example             <- Variables de entorno (sin valores reales)
|-- docs/
|   |-- entrega-1a.pdf       <- Este informe
|   |-- diagramas/
|       |-- c4_nivel1.png
|       |-- c4_nivel2.png
|       |-- diagrama_mer.png
|       |-- wireframe_*.png
|   |-- adr/
|       |-- ADR-001-tecnologia.md
|-- database/
|   |-- schema.sql           <- DDL completo de las 24 tablas
|   |-- seed_data.sql        <- Datos iniciales de tablas maestras
|-- backend-springboot/      <- Proyecto Spring Boot (Maven / Gradle)
|-- frontend-angular/        <- Proyecto Angular (npm)
```

Referencias Bibliográficas

- [1] V. G. R. Liabor. Enhancing library services through digital cataloging techniques. *International Journal of Advanced Research in Science, Communication and Technology*, pages 516–526, 2023. URL: <https://doi.org/10.48175/ijarsct-12989>.

- [2] M. N. Shivaji. Intelligent library management system. *Trends in Computer Science and Information Technology*, 9(1):001–009, 2024. URL: <https://doi.org/10.17352/tcsit.000074>.
- [3] V. Aienobe and M. Z. Iqbal. Responsive web design for enhanced user experience (ux) and user interface (ui). *International Journal of Computer Applications*, 187(34):72–93, 2025. URL: <https://doi.org/10.5120/ijca2025925533>.
- [4] T.-J. Ng, K.-W. Ng, and S.-C. Haw. Lib-bot: A smart librarian-chatbot assistant. *International Journal of Computing and Digital Systems*, 16(1):1–11, 2024. URL: <https://doi.org/10.12785/ijcds/160101>.
- [5] A. J. Adetayo. Artificial intelligence chatbots in academic libraries: The rise of chatgpt. *Library Hi Tech News*, 2023. URL: <https://doi.org/10.1108/LHTN-01-2023-0007>.
- [6] D. Zowghi and C. Coulin. Requirements elicitation: A survey of techniques, approaches, and tools. In A. Aurum and C. Wohlin, editors, *Engineering and Managing Software Requirements*, pages 19–46. Springer, 2005. URL: https://doi.org/10.1007/3-540-28244-0_2.
- [7] M. Bano, D. Zowghi, and F. da Cunha. Teaching requirements elicitation interviews: An empirical study of learning from mistakes. *Requirements Engineering*, 24:259–289, 2019. URL: <https://doi.org/10.1007/s00766-019-00313-0>.
- [8] I. Sommerville. *Ingeniería del software*. Pearson Education, 10 edition, 2015.
- [9] K. Stine et al. Guidelines for the selection, configuration, and use of transport layer security (tls) implementations. Technical Report NIST SP 800-52 Rev. 2, National Institute of Standards and Technology, 2019. URL: <https://doi.org/10.6028/NIST.SP.800-52r2>.
- [10] F. Wayesa, G. Guyo, M. Meko, and M. I. Uddin. Pattern-based hybrid book recommendation system using semantic relationships. *Scientific Reports*, 13:3693, 2023. URL: <https://doi.org/10.1038/s41598-023-30987-0>.
- [11] P. Hu and Y. Zhang. Big data analytics in library services with ai: Personalized content recommendations and catalog optimization. *IEEE Access*, 13:88412–88420, 2025. URL: <https://doi.org/10.1109/ACCESS.2025.3570200>.
- [12] A. Parlakkı hç. Evaluating the effects of responsive design on the usability of academic websites in the pandemic. *Education and Information Technologies*, 27(1):1307–1322, 2022. URL: <https://doi.org/10.1007/s10639-021-10650-9>.
- [13] P. Meesad and A. Mingkhwan. Ai-powered smart digital libraries. In *Libraries in Transformation*, volume 157, pages 199–228. Springer, 2024. URL: https://doi.org/10.1007/978-3-031-69216-1_10.
- [14] T. Rajčić, D. Boberić-Krstić, D. Tešendić, and G. Milosavljević. Validation of gdpr compliance in a library management system: A bisis and temda case study. *Information Technology and Libraries*, 44(4), 2025. URL: <https://doi.org/10.5860/ital.v44i4.17407>.
- [15] L. Bass, P. Clements, and R. Kazman. *Software architecture in practice*. Addison-Wesley, 4 edition, 2021.

- [16] S. Brown. *The C4 model for visualising software architecture*. Leanpub, 2023. URL: <https://c4model.com/>.
- [17] M. Fowler. *Patterns of enterprise application architecture*. Addison-Wesley, 2002.
- [18] C. Walls. *Spring in action*. Manning Publications, 6 edition, 2022.
- [19] L. Ayinde, R. Ebiefung, and B. D. Oladokun. Adoption of artificial intelligence in academic libraries: A systematic review of current practices, challenges, and research opportunities. *The Journal of Academic Librarianship*, 52(1):103185, 2026. URL: <https://doi.org/10.1016/j.acalib.2025.103185>.